

Software Design Specification (SDS)

Form Claw — Automated PDF Form-Filling System

<<<<<< HEAD

Version: 1.1

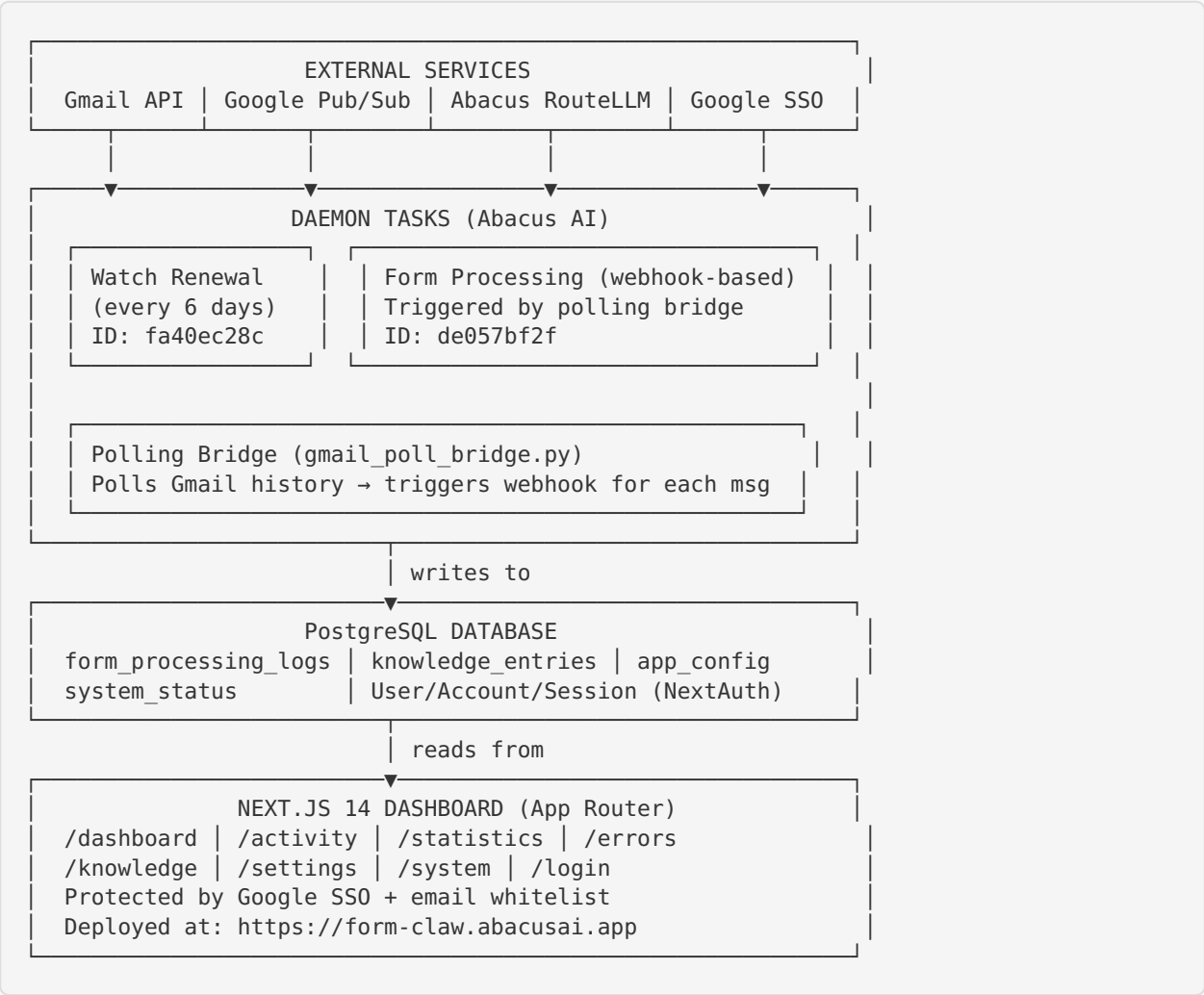
=====

Version: 1.2

72c35ee (v1.2: Add polling/webhook toggles, security filter, update docs)
Date: June 4, 2026
Author: Form Claw Project

1. Architecture Overview

The system follows a **daemon + webhook + dashboard** architecture:



2. Technology Stack

Component	Technology	Version
Dashboard Framework	Next.js (App Router)	14
Language	TypeScript	5.x
ORM	Prisma	6.7.0
Database	PostgreSQL	(Abacus AI hosted)
Authentication	NextAuth.js + Google OAuth	4.x
UI Components	shadcn/ui + Tailwind CSS	Latest
Charts	Recharts	Latest
Icons	Lucide React	Latest
Daemon Runtime	Abacus AI Agent daemon tasks	-
Processing Scripts	Python 3	3.x
PDF Manipulation	Python (PyPDF2/reportlab) + LLM	-
OCR	Abacus RouteLLM (vision)	-
Email	Gmail API (OAuth2)	v1
Notifications	Google Cloud Pub/Sub	-
DOC Conversion	LibreOffice (headless)	-
Image Merging	img2pdf (Python)	-

3. Database Schema

3.1 Tables

`form_processing_logs`

Tracks every form processing event.

```

id                SERIAL PRIMARY KEY
email_message_id  TEXT UNIQUE
received_at       TIMESTAMPTZ
sender_email      TEXT
sender_name       TEXT
subject          TEXT
attachment_filename TEXT
attachment_type   TEXT
attachment_count  INT DEFAULT 0
page_count        INT
target_person     TEXT
signer            TEXT
processing_status  TEXT DEFAULT 'processing' -- processing|completed|failed
processing_started_at TIMESTAMPTZ
processing_completed_at TIMESTAMPTZ
processing_time_seconds DECIMAL(10,2)
filled_pdf_filename TEXT
error_message     TEXT
error_type        TEXT
instructions_detected TEXT
marked_as_read    BOOLEAN DEFAULT false
created_at        TIMESTAMPTZ DEFAULT now()
updated_at        TIMESTAMPTZ DEFAULT now()

```

Indexes: `idx_fpl_created_at`, `idx_fpl_processing_status`, `idx_fpl_received_at`,
`idx_fpl_sender_email`

knowledge_entries

Persistent knowledge store for family data, auto-curated from interactions.

```

id                TEXT PRIMARY KEY (cuid)
category          TEXT DEFAULT 'general' -- personal|address|medical|school|contact|
                 preference|general
key              TEXT
value            TEXT
source           TEXT DEFAULT 'manual' -- manual|initial_seed|email_from_xxx
language         TEXT DEFAULT 'both' -- en|he|both
applies_to_person TEXT -- NULL=family-wide, or person name
is_active        BOOLEAN DEFAULT true
created_at       TIMESTAMPTZ DEFAULT now()
updated_at       TIMESTAMPTZ DEFAULT now()

```

Indexes: `category`, `appliesToPerson`, `isActive`

app_config

Key-value configuration store.

```

id                TEXT PRIMARY KEY (cuid)
key              TEXT UNIQUE
value            TEXT
label            TEXT
category         TEXT DEFAULT 'general' -- fonts|general
created_at       TIMESTAMPTZ DEFAULT now()
updated_at       TIMESTAMPTZ DEFAULT now()

```

Index: `category`

system_status

<<<<<< HEAD

Gmail watch status tracking.

=====

Gmail watch status + intake channel control.

 72c35ee (v1.2: Add polling/webhook toggles, security filter, update docs)

```

id                TEXT PRIMARY KEY (cuid)
gmail_watch_active BOOLEAN DEFAULT false
watch_expiration  TIMESTAMPTZ
last_watch_renewal TIMESTAMPTZ
last_successful_form TIMESTAMPTZ
<<<<<< HEAD
=====
polling_enabled    BOOLEAN DEFAULT false
webhook_enabled    BOOLEAN DEFAULT true
>>>>>> 72c35ee (v1.2: Add polling/webhook toggles, security filter, update docs)
created_at         TIMESTAMPTZ DEFAULT now()
updated_at         TIMESTAMPTZ DEFAULT now()

```

NextAuth tables

User , Account , Session , VerificationToken — standard NextAuth.js schema with Prisma adapter.

4. API Endpoints

Method	Endpoint	Description	Auth
GET	/api/logs	Paginated form processing logs with filters	Yes
GET	/api/stats	Overall statistics (30 days)	Yes
GET	/api/stats/range	Statistics for custom date range	Yes
GET	/api/errors	Paginated error logs with type filter	Yes
GET	/api/errors/export	CSV export of all errors	Yes
<<<<<<< HEAD			
GET	/api/system	System health status	Yes
=====			
GET	/api/system	System health status + toggle state	Yes
PATCH	/api/system	Toggle pollingEnabled / webhookEnabled	Yes
>>>>>>> 72c35ee (v1.2: Add polling/webhook toggles, security filter, update docs)			
GET	/api/config	App configuration entries	Yes
PUT	/api/config	Update configuration entries	Yes
GET	/api/knowledge	Knowledge entries (paginated, filterable)	Yes
POST	/api/knowledge	Create new knowledge entry	Yes

Method	Endpoint	Description	Auth
PUT	/api/knowledge/:id	Update knowledge entry	Yes
DELETE	/api/knowledge/:id	Soft-delete knowledge entry (isActive=false)	Yes
POST/GET	/api/auth/[...nextauth]	NextAuth.js authentication	-
POST	/api/auth/login	Credentials login	-

4.1 Query Parameters

GET /api/logs:

- `page` (default: 1), `limit` (default: 20)
- `status` (filter: completed|failed|processing)
- `sender` (email filter)
- `search` (subject text search)
- `startDate` , `endDate` (date range on received_at)

GET /api/knowledge:

- `page` (default: 1), `limit` (default: 50)
- `category` (filter category)
- `person` (filter appliesToPerson; "family" = null entries)
- `search` (text search on key + value)

5. Daemon Tasks

5.1 Gmail Watch Renewal (ID: `fa40ec28c`)

- **Schedule:** Every 6 days
- **Script:** `/home/ubuntu/shared/gmail_watch_renew.py`
- **Function:** Calls Gmail API `watch()` to renew Pub/Sub subscription
- **State file:** `/home/ubuntu/shared/gmail_watch_state.json`
- **Pub/Sub config:** Project `form-claw` , Topic `gmail-formbot`

5.2 Form Processing (ID: `de057bf2f`)

- **Trigger:** Webhook (event-based, triggered by polling bridge)
- **Webhook URL:** `https://apps.abacus.ai/api/webhooks?abacus_deployment_token=...`
- **Processing pipeline:**
 1. Receive webhook with `message_id` , `thread_id` , `sender` , `subject`
 2. Fetch full email via Gmail API
 3. Extract attachments
 4. Normalize: PDF direct, DOC→PDF via LibreOffice, Images→PDF via img2pdf
 5. If image-based: OCR via LLM vision API

6. Analyze form structure via LLM
7. Map family data fields from knowledge store + family_data.json
8. Parse instructions from subject/body for target child and signer
9. Fill PDF with appropriate font (Playzone for EN, FtPilKahol2 for HE)
10. Place signature (Ze'ev or Keren) at detected signature locations
11. Reply in thread with filled PDF
12. Log event to `form_processing_logs` table
13. Mark email as READ

5.3 Polling Bridge

- **Script:** `/home/ubuntu/shared/gmail_poll_bridge.py`
- **Function:** Polls Gmail history API for new messages since last check, filters by whitelist, triggers webhook for each new message
- **State file:** `/home/ubuntu/shared/gmail_history_state.json`

<<<<<<< HEAD

5.4 Security Filter

- **Script:** `/home/ubuntu/shared/security_filter.py`
- **Function:** Scans email subject, body, and attachment text for prompt injection patterns before LLM processing
- **Pattern categories:** instruction override, role hijacking, data exfiltration, encoded payloads, form-fill manipulation, signature forgery
- **Bilingual:** Detects patterns in both English and Hebrew
- **Risk scoring:** 0.0–1.0, blocks at ≥ 0.70
- **Usage:** `from security_filter import scan_email_content; verdict = scan_email_content(subject, body, attachment_text)`
- **CLI:** `python3 security_filter.py "text to scan"` — exits 1 if blocked, 0 if clean

>>>>>> 72c35ee (v1.2: Add polling/webhook toggles, security filter, update docs)

6. File Structure

```

/home/ubuntu/form_filler_dashboard/
├── nextjs_space/
│   ├── app/
│   │   ├── api/
│   │   │   ├── auth/[...nextauth]/route.ts # NextAuth handler
│   │   │   ├── auth/login/route.ts # Credentials login
│   │   │   ├── config/route.ts # GET/PUT app config
│   │   │   ├── errors/route.ts # GET error logs
│   │   │   ├── errors/export/route.ts # GET CSV export
│   │   │   ├── knowledge/route.ts # GET/POST knowledge
│   │   │   ├── knowledge/[id]/route.ts # PUT/DELETE knowledge entry
│   │   │   ├── logs/route.ts # GET processing logs
│   │   │   ├── stats/route.ts # GET statistics
│   │   │   ├── stats/range/route.ts # GET stats by date range
│   │   │   └── system/route.ts # GET system status
│   │   ├── activity/_components/activity-client.tsx
│   │   ├── activity/page.tsx
│   │   ├── dashboard/_components/dashboard-client.tsx
│   │   ├── dashboard/page.tsx
│   │   ├── errors/_components/errors-client.tsx
│   │   ├── errors/page.tsx
│   │   ├── knowledge/_components/knowledge-client.tsx
│   │   ├── knowledge/page.tsx
│   │   ├── login/page.tsx
│   │   ├── settings/_components/settings-client.tsx
│   │   ├── settings/page.tsx
│   │   └── statistics/_components/*.tsx # Chart components
│   │       ├── statistics/page.tsx
│   │       ├── system/_components/system-client.tsx
│   │       ├── system/page.tsx
│   │       ├── globals.css
│   │       ├── layout.tsx
│   │       ├── page.tsx # Root redirect
│   │       └── providers.tsx # SessionProvider wrapper
│   ├── components/
│   │   ├── layouts/
│   │   │   ├── sidebar.tsx # Main navigation sidebar
│   │   │   ├── dashboard-layout.tsx
│   │   │   └── ...other layout components
│   │   └── ui/ # shadcn/ui components
│   ├── lib/
│   │   ├── auth.ts # NextAuth configuration
│   │   ├── prisma.ts # Prisma client singleton
│   │   ├── db.ts # Legacy Prisma export
│   │   └── utils.ts # Utility functions
│   ├── prisma/
│   │   └── schema.prisma # Database schema
│   ├── scripts/
│   │   └── seed.ts # Database seeding
│   ├── public/
│   │   └── fonts/
│   │       ├── Playzone.ttf
│   │       └── FtPilKahol2.ttf
│   ├── middleware.ts # Auth + whitelist middleware
│   ├── .env # Environment variables
│   ├── SRS.md # This document
│   ├── SDS.md # Design specification
│   ├── next.config.js
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   └── package.json

```

```

/home/ubuntu/shared/
├── family_data.json           # Legacy family data (now in KB)
├── gmail_poll_bridge.py      # Polling bridge script
├── gmail_watch_renew.py      # Watch renewal script
├── gmail_history_state.json  # Polling state
├── gmail_watch_state.json   # Watch state
<<<<<<< HEAD
=====
├── security_filter.py        # Prompt injection security filter
├── send_plain_reply.py      # Plain-text MIME reply helper
>>>>>> 72c35ee (v1.2: Add polling/webhook toggles, security filter, update docs)
├── fonts/
│   ├── Playzone.ttf
│   └── FtPilKahol2.ttf
├── keren sig.png            # Keren's signature
├── zr signature nxp.png     # Ze'ev's signature
└── db/
    ├── schema.sql          # Raw SQL schema
    └── log_form_event.py    # DB logging helper

```

7. Authentication & Security

7.1 Dashboard Authentication

- **Provider:** Google OAuth 2.0 via NextAuth.js
- **Strategy:** JWT sessions
- **Whitelist enforcement:** Both in middleware.ts and lib/auth.ts
- **Whitelisted emails:**
 - k6622024@gmail.com
 - 2396119@gmail.com
 - zeev@infiniplex.life
 - targetmailbox@gmail.com
 - russakbot@gmail.com
 - john@doe.com (test account)
- **Login page:** /login with Google SSO button
- **Credentials fallback:** Email/password for test account

7.2 API Security

- All API routes check `getSession(authOptions)` → 401 if unauthorized
- Middleware protects page routes and API routes matching configured patterns

7.3 Gmail API Security

- OAuth2 credentials stored in `/home/ubuntu/.config/abacusai_auth_secrets.json`
 - Token refresh handled by the polling bridge
-

8. Environment Variables

```

DATABASE_URL=postgresql://...           # PostgreSQL connection string
NEXTAUTH_SECRET=...                     # JWT signing secret
GOOGLE_CLIENT_ID=...                    # Google OAuth client ID
GOOGLE_CLIENT_SECRET=...                # Google OAuth client secret
# NEXTAUTH_URL is auto-configured by Abacus AI

```

9. Deployment

- **Platform:** Abacus AI Agent
- **Build:** Next.js standalone output mode
- **URL:** `https://form-claw.abacusai.app`
- **Database:** Abacus AI hosted PostgreSQL (shared dev/prod)
- **CI/CD:** Checkpoint-based deployment via Abacus AI platform
- **GitHub:** `zeevrussak/form-claw` for version control

10. Step-by-Step Recreation Guide for LLM Agent

To recreate this project from scratch:

Phase 1: Dashboard Setup

1. **Initialize** a Next.js 14 project with App Router, TypeScript, Tailwind CSS
2. **Install dependencies:** `prisma`, `@prisma/client`, `next-auth`, `@next-auth/prisma-adapter`, `bcryptjs`, `recharts`, `lucide-react`, `shadcn/ui` components
3. **Initialize PostgreSQL database** (Abacus AI hosted)
4. **Create Prisma schema** with all models from Section 3
5. **Run** `prisma db push` to create tables
6. **Set up NextAuth** with Google OAuth + Credentials providers (see `lib/auth.ts`)
7. **Create middleware** with email whitelist (see `middleware.ts`)
8. **Build API routes** for logs, stats, errors, system, config, knowledge (see Section 4)
9. **Build pages:** login, dashboard, activity, statistics, errors, system, knowledge, settings
10. **Create sidebar navigation** with all page links
11. **Run seed script** to populate initial data
12. **Deploy** to Abacus AI

Phase 2: Email Processing Setup

1. **Configure Gmail API** OAuth credentials for `russakbot@gmail.com`
2. **Set up Google Cloud Pub/Sub** topic `gmail-formbot` in project `form-claw`
3. **Grant Pub/Sub publish rights** to Gmail service account
4. **Create Gmail watch** on INBOX label pointing to Pub/Sub topic
5. **Write polling bridge** (`gmail_poll_bridge.py`) that polls Gmail history and triggers webhooks

6. Write form processing daemon that:

- Fetches email + attachments via Gmail API
- Normalizes to PDF (LibreOffice for DOC, img2pdf for images)
- OCRs image-based PDFs via LLM vision
- Analyzes form structure via LLM
- Maps family data from knowledge store
- Fills PDF with configured fonts
- Places correct signature
- Replies in thread
- Logs to database

7. Create watch renewal daemon (every 6 days)

8. Upload font files (Playzone.ttf, FtPilKahol2.ttf) and signature images

Phase 3: Knowledge & Configuration

1. **Seed knowledge base** with all family data from `family_data.json`
2. **Seed font config** (default: Playzone EN, FtPilKahol2 HE)
3. **Implement auto-curation** in form processing daemon: when new info is learned, write to `knowledge_entries` table
4. **Test end-to-end:** send email → receive filled form → check dashboard logs

Key Implementation Notes for LLM Agent:

- Use `export const dynamic = 'force-dynamic'` on all pages/routes reading env vars
- Use `@/lib/prisma` singleton for all DB access
- All API routes must check `getSession(authOptions)` and return 401 if unauthorized
- Dashboard styling: dark theme (slate-900/950 gradients), glassmorphism cards, blue/purple/amber accents
- Charts use Recharts with dynamic imports for code splitting
- Knowledge soft-delete (set `isActive=false`) rather than hard delete
- Font config stored in `app_config` table, read by daemon before each form fill
- Signature files: `zr signature nxp.png` (Ze'ev), `keren sig.png` (Keren)
- Date format preference: DD-MM-YYYY
- Remember: CLIL with a C, not K